Boletín de ejercicios: interfaces funcionales, lambdas, streams y Optional en Java

Boletín de ejercicios: interfaces funcionales, lambdas, streams y Optional en Java **1**
Bloque 0. Recomendaciones para el alumnado 4
Nivel 1. Muy básicos **5**
Ejercicio 1 5
Ejercicio 2 5
Ejercicio 3 6
Ejercicio 4 6
Ejercicio 5 7
Ejercicio 6 7
Nivel 2. Básicos con código **8**
Ejercicio 7 8
Ejercicio 8 8
Ejercicio 9 8
Ejercicio 10 9
Ejercicio 11 9
Ejercicio 12 9
Nivel 3. Introducción a streams **10**
Ejercicio 13 10
Ejercicio 14 10
Ejercicio 15 10
Ejercicio 16 11
Ejercicio 17 11
Ejercicio 18 11
Nivel 4. Operaciones intermedias **12**
Ejercicio 19 12
Ejercicio 20 12
Ejercicio 21 12
Ejercicio 22 12
Ejercicio 23 13
Ejercicio 24 13
Ejercicio 25 13
Ejercicio 26 14
Nivel 5. Operaciones terminales **14**
Ejercicio 27 14

Ejercicio 28 15
Ejercicio 29 15
Ejercicio 30 15
Ejercicio 31 16
Nivel 6. Optional 16
Ejercicio 32 16
Ejercicio 33 17
Ejercicio 34 17
Ejercicio 35 17
Ejercicio 36 18
Ejercicio 37 18
Nivel 7. Relación entre streams e interfaces funcionales 18
Ejercicio 38 18
Ejercicio 39 19
Ejercicio 40 19
Ejercicio 41 20
Nivel 8. Problemas de aplicación 20
Ejercicio 42 21
Ejercicio 43 21
Ejercicio 44 21
Ejercicio 45 22
Ejercicio 46 22
Ejercicio 47 23
Nivel 9. Ejercicios sobre razonamiento y errores frecuentes 23
Ejercicio 48 23
Ejercicio 49 23
Ejercicio 50 24
Ejercicio 51 24
Ejercicio 52 24
Nivel 10. Ejercicios largos / integradores 25
Ejercicio 53 25
Ejercicio 54 26
Ejercicio 55 26
Ejercicio 56 26

Ejercicio 57 27
Ejercicio 58 27
Nivel 11. Retos 28
Reto 1 28
Reto 2 28
Reto 3 28
Reto 4 29
Reto 5 29

Bloque 0. Recomendaciones para el alumnado

Antes de empezar:

- intenta resolver cada ejercicio primero “a mano”, pensando qué tipo de dato entra y qué tipo sale;
- fíjate en si estás ante una condición, una transformación, un consumo o una generación de valores;
- en streams, distingue siempre entre:
  - fuente;
  - operaciones intermedias;
  - operación terminal;
- en Optional, piensa siempre que puede haber valor o no haberlo.

Nivel 1. Muy básicos

Ejercicio 1

Indica qué hace cada una de estas interfaces funcionales estándar de Java:

- Predicate<T> Recibe un valor y devolve un booleano.
- Function<T, R> Transforma un valor de tipo T en tipo R.
- Consumer<T> Recibe un valor de T y no devolve nada
- Supplier<T> Suministra un valor sin recibir argumentos

Para cada una, explica:

- qué recibe;
- qué devuelve;
- para qué tipo de tarea suele usarse.

Ejercicio 2

Relaciona cada situación con la interfaz funcional más adecuada:

- comprobar si un número es par;

```
Predicate<T>  t  = x -> x == y;
```

- transformar un texto a mayúsculas;

```
Function<T,R> num = (x,y) -> x.toString();
```

- imprimir un mensaje por pantalla;

```
Consumer<T> res = System.out::println;
```

- generar un número aleatorio;

```
Supplie<Integer> valorAleatorio =  ()-> Math.random()*100;
```

- comparar dos cadenas por longitud.

```
Compator<String> mayor = (a,b) -> a.lenght > b.lenght;
```

Ejercicio 3

Escribe una lambda para cada caso:

- una lambda que reciba un String y devuelva su longitud;
- una lambda que reciba un Integer y diga si es positivo;
- una lambda que reciba un String y lo imprima;
- una lambda que no reciba nada y devuelva "Hola".

Ejercicio 4

Dada esta interfaz funcional:

```
@FunctionalInterface
public interface Operacion {
   int aplicar(int a, int b);
}
```

Escribe una lambda para:

- sumar dos enteros;

```
Operacion sumar = (a,b)-> a+b;
int res = sumar.aplicar(4,5)
```

- restar dos enteros;
  Operacion in
- multiplicar dos enteros.

Ejercicio 5

Explica por qué esta expresión es válida:

```
Predicate<String> p = s -> s.isBlank();
```

Debes explicar:

- qué tipo espera Java en el lado izquierdo;
- qué método abstracto de la interfaz funcional está implicado;
- cómo interpreta Java esa lambda.

Ejercicio 6

Indica cuáles de estas variables pueden ser capturadas por una lambda y cuáles no. Justifica la respuesta.

Caso A:

```
int x = 5;
Function<Integer, Integer> f = n -> n + x;
```

Caso B:

```
int x = 5;
x = 7;
Function<Integer, Integer> f = n -> n + x;
```

Caso C:

```
final int x = 5;
Function<Integer, Integer> f = n -> n + x;
```

Nivel 2. Básicos con código

Ejercicio 7

Crea un Predicate<Integer> que compruebe si un número es múltiplo de 3.

Después, úsalo con varios valores y muestra el resultado por pantalla.

Ejercicio 8

Crea un Function<String, String> que reciba una cadena y devuelva esa misma cadena rodeada por corchetes.

Ejemplo:

- entrada: "hola"
- salida: "[hola]"

Ejercicio 9

Crea un Consumer<String> que imprima por pantalla el texto recibido precedido por "Mensaje: ".

Ejercicio 10

Crea un Supplier<Integer> que devuelva siempre el número 42.

Después crea otro que devuelva un número aleatorio entre 1 y 10.

Ejercicio 11

Dada esta lista:

```
List<String> nombres = List.of("Ana", "Luis", "Marta",
"Alberto");
```

Crea una lambda o referencia a método para:

- obtener la longitud de cada nombre;
- convertir cada nombre a mayúsculas;
- comprobar si empieza por la letra A.

Ejercicio 12

Escribe el equivalente con lambda de esta clase anónima:

```
Comparator<String> c = new Comparator<String>() {
   @Override
   public int compare(String a, String b) {
      return Integer.compare(a.length(), b.length());
   }
};
```

Nivel 3. Introducción a streams

Ejercicio 13

Crea un stream vacío de cadenas.

Después, conviértelo en una lista.

Ejercicio 14

Crea un stream a partir de estos valores:

- "rojo"
- "verde"
- "azul"

Después recórrelo e imprime sus elementos.

Ejercicio 15

Dada esta lista:

```
List<Integer> numeros = List.of(1, 2, 3, 4, 5, 6);
```

Crea un stream a partir de ella y obtén:

- solo los números pares;
- la cantidad de números que hay;

- una nueva lista con cada número multiplicado por 10.

Ejercicio 16

Dado este array:

```
String[] frutas = {"pera", "manzana", "uva", "plátano"};
```

Crea un stream y obtén una lista con las frutas en mayúsculas.

Ejercicio 17

Usa IntStream.range o IntStream.rangeClosed para generar:

- los números del 1 al 5;
- los números del 0 al 10;
- los números del 10 al 20.

Ejercicio 18

Explica la diferencia entre:

- Stream.empty()
- Stream.of(...)

- coleccion.stream()

Nivel 4. Operaciones intermedias

Ejercicio 19

Dada esta lista:

```
List<String> palabras = List.of("sol", "montaña", "mar",
"luz", "programación");
```

Obtén una nueva lista con las palabras de más de 3 letras.

Ejercicio 20

Con la misma lista del ejercicio anterior, obtén una lista con todas las palabras en mayúsculas.

Ejercicio 21

Con la misma lista, obtén una lista con las longitudes de todas las palabras.

Ejercicio 22

Dada esta lista:

```
List<Integer> numeros = List.of(1, 2, 2, 3, 4, 4, 5, 5, 5);
```

Obtén una lista sin elementos repetidos.

Ejercicio 23

Dada esta lista:

```
List<String> nombres = List.of("Luis", "Ana", "Marta",
"Pablo");
```

Obtén:

- una lista ordenada alfabéticamente;
- una lista ordenada por longitud de menor a mayor.

Ejercicio 24

Dada esta lista:

```
List<Integer> numeros = List.of(10, 20, 30, 40, 50, 60);
```

Obtén:

- los tres primeros elementos;
- todos excepto los dos primeros.

Usa limit y skip.

Ejercicio 25

Explica qué diferencia hay entre map y filter.

Pon un ejemplo de cada uno.

Ejercicio 26

Dada esta lista:

```
List<String> frases = List.of("hola mundo", "java streams",
"programación funcional");
```

Obtén una lista con todas las palabras individuales usando flatMap.

Resultado esperado aproximado:

```
["hola", "mundo", "java", "streams", "programación",
"funcional"]
```

Nivel 5. Operaciones terminales

Ejercicio 27

Dada esta lista:

```
List<Integer> numeros = List.of(2, 4, 6, 8, 10);
```

Calcula:

- cuántos números hay;
- si hay algún número mayor que 7;
- si todos son pares;

- si ninguno es negativo.

Ejercicio 28

Dada esta lista:

```
List<String> nombres = List.of("Ana", "Luis", "Marta",
"Pablo");
```

Usa forEach para imprimir cada nombre.

Después, intenta hacerlo con referencia a método si es posible.

Ejercicio 29

Dada esta lista:

```
List<Integer> numeros = List.of(1, 2, 3, 4, 5);
```

Usa reduce para obtener:

- la suma total;
- el producto total;
- el mayor valor de la lista.

Ejercicio 30

Dada esta lista:

```
List<String> palabras = List.of("sol", "mar", "sol", "luna",
"mar", "estrella");
```

Usa collect para obtener:

- una lista;
- un conjunto (Set);
- un mapa que agrupe las palabras por longitud.

Ejercicio 31

Dada esta lista:

```
List<String> nombres = List.of("Ana", "Luis", "Marta",
"Pablo", "Alberto");
```

Obtén:

- el primer nombre que empiece por A;
- cualquier nombre que tenga más de 5 letras.

Piensa qué tipo devuelven findFirst y findAny.

Nivel 6. Optional

Ejercicio 32

Explica con tus palabras qué representa Optional<T> y qué problema intenta evitar.

Ejercicio 33

Crea:

- un Optional<String> que contenga "Hola";
- un Optional<String> vacío;
- un Optional<String> a partir de una variable que puede ser null.

Ejercicio 34

Dado este Optional<String>:

```
Optional<String> nombre = Optional.of("Marta");
```

Haz lo siguiente:

- imprime el valor solo si está presente;
- obtén su longitud usando map;
- filtra para quedarte con él solo si empieza por M.

Ejercicio 35

Dado este código:

```
Optional<String> nombre = Optional.empty();
```

Obtén un valor por defecto:

- usando orElse("Desconocido");
- usando orElseGet(...).

Explica la diferencia entre ambos.

Ejercicio 36

Dado un Optional<String>, transforma su contenido a mayúsculas si existe. Si no existe, devuelve "SIN VALOR".

Ejercicio 37

¿Por qué no es buena idea abusar de get() en Optional?

Pon un ejemplo donde pueda producirse un error.

Nivel 7. Relación entre streams e interfaces funcionales

Ejercicio 38

Relaciona cada operación de stream con la interfaz funcional que suele usar:

- filter
- map

- forEach
- sorted(Comparator)
- reduce

Ejercicio 39

Para cada una de estas expresiones, indica:

- qué tipo de interfaz funcional interviene;
- qué recibe;
- qué devuelve.

A)

```
.filter(s -> s.length() > 3)
```

B)

```
.map(String::toUpperCase)
```

C)

```
.forEach(System.out::println)
```

D)

```
.reduce(0, Integer::sum)
```

Ejercicio 40

Explica por qué esta tubería de stream no se ejecuta todavía:

```
Stream<String> s = nombres.stream()
      .filter(x -> x.length() > 3)
      .map(String::toUpperCase);
```

¿Qué falta para que realmente procese los datos?

Ejercicio 41

Indica cuáles de estas operaciones son intermedias y cuáles terminales:

- filter
- map
- sorted
- count
- forEach
- toList
- limit
- findFirst

Nivel 8. Problemas de aplicación

Ejercicio 42

Dada esta lista de números:

```
List<Integer> numeros = List.of(3, 8, 1, 9, 2, 10, 7, 6);
```

Obtén una lista con los números pares, ordenados de menor a mayor y multiplicados por 2.

Ejercicio 43

Dada esta lista:

```
List<String> nombres = List.of("ana", "luis", "marta",
"alberto", "eva");
```

Obtén una lista con los nombres:

- cuya longitud sea mayor o igual que 4;
- en mayúsculas;
- ordenados alfabéticamente.

Ejercicio 44

Dada esta lista de frases:

```
List<String> frases = List.of(
   "Java es potente",
   "Los streams son útiles",
   "Optional evita muchos null"
);
```

Obtén:

- todas las palabras individuales;
- solo las palabras de más de 4 letras;
- una lista ordenada alfabéticamente sin repetidos.

Ejercicio 45

Dada esta lista:

```
List<Integer> numeros = List.of(5, 12, 7, 20, 3, 18);
```

Calcula:

- la suma de todos los números;
- el mayor;
- si existe algún número mayor que 15;
- cuántos son impares.

Ejercicio 46

Dada esta lista:

```
List<String> nombres = List.of("Ana", "Luis", "Marta",
"Alberto", "Lucía", "Andrés");
```

Agrupa los nombres por su inicial. El resultado debe ser un Map<Character, List<String>>.

Ejercicio 47

Dada una lista de cadenas, obtén la palabra más larga usando streams.

Si la lista está vacía, el resultado debe gestionarse con Optional.

Nivel 9. Ejercicios sobre razonamiento y errores frecuentes

Ejercicio 48

Analiza este código y explica qué error conceptual tiene:

```
List<String> nombres = List.of("Ana", "Luis", "Marta");

Stream<String> s = nombres.stream();

s.forEach(System.out::println);

s.forEach(System.out::println);
```

Ejercicio 49

Analiza este código y explica por qué falla:

```
int total = 0;

List<Integer> numeros = List.of(1, 2, 3);

numeros.forEach(n -> total += n);
```

Relaciona tu explicación con el concepto de variable efectivamente final.

Ejercicio 50

Analiza este uso de Optional:

```
Optional<String> nombre = buscarNombre();

if (nombre.isPresent()) {

   System.out.println(nombre.get().toUpperCase());

}
```

Reescríbelo de una forma más idiomática usando operaciones de Optional.

Ejercicio 51

Explica por qué este uso de map no es correcto si lo que queremos es quedarnos solo con los números pares:

```
List<Integer> resultado = numeros.stream()

      .map(n -> n % 2 == 0)

      .toList();
```

¿Qué operación habría que usar en su lugar?

Ejercicio 52

Explica qué diferencia hay entre estas dos expresiones:

```
.map(s -> s.length())
```

y

```
.filter(s -> s.length() > 3)
```

Nivel 10. Ejercicios largos / integradores

Ejercicio 53

Se tiene una lista de alumnos representados como cadenas con este formato:

```
List<String> alumnos = List.of(
   "Ana:7",
   "Luis:5",
   "Marta:9",
   "Pablo:4",
   "Lucía:8"
);
```

Realiza las siguientes tareas con streams:

- obtener solo los nombres;
- obtener solo las notas;
- calcular la nota media;
- obtener los alumnos aprobados;
- obtener el nombre del primer alumno con nota mayor o igual que 9.

Ejercicio 54

Dada una lista de palabras, construye una tubería de stream que:

- elimine las palabras repetidas;
- descarte las que tengan menos de 4 letras;
- convierta las restantes a minúsculas;
- las ordene por longitud;
- las recoja en una lista final.

Ejercicio 55

Dada una lista de listas de enteros, construye una solución con streams que:

- convierta todo en un único stream;
- elimine repetidos;
- deje solo los mayores que 10;
- calcule la suma total.

Ejercicio 56

Implementa un método que reciba una lista de cadenas y devuelva un Optional<String> con la cadena más corta.

Si la lista está vacía, debe devolver Optional.empty().

Ejercicio 57

Implementa un método que reciba una lista de enteros y devuelva un Map<Boolean, List<Integer>> donde:

- true corresponda a los pares;
- false corresponda a los impares.

Pista: investiga una forma de particionar con Collectors.

Ejercicio 58

Diseña un ejemplo realista donde tenga sentido usar:

- Predicate
- Function
- Consumer
- Supplier
- Stream
- Optional

No hace falta implementar un programa completo, pero sí describir claramente el caso y justificar por qué usarías cada uno.

Nivel 11. Retos

Reto 1

Sin usar bucles explícitos, toma una lista de cadenas y obtén una lista con las iniciales en mayúsculas de aquellas palabras que tengan al menos 5 letras.

Ejemplo:

- entrada: ["casa", "montaña", "sol", "programación"]
- salida: ["M", "P"]

Reto 2

Dada una lista de frases, obtén un mapa donde la clave sea el número de palabras de cada frase y el valor sea la lista de frases que tienen esa cantidad de palabras.

Reto 3

Crea una tubería de stream que, a partir de los números del 1 al 100:

- deje solo los múltiplos de 3;
- elimine los que sean también múltiplos de 2;

- transforme cada número en una cadena del tipo "Número: X";
- tome solo los 10 primeros;
- los imprima.

Reto 4

Dada una lista de nombres, obtén un Optional<String> con el nombre más largo que empiece por vocal.

Reto 5

Construye un ejemplo en el que una lambda capture una variable externa válida y otro en el que no sea válida. Explica por qué en cada caso.